Unit I

Introduction to DBMS and Data Models

Syllabus

- Introduction,
- Purpose of Database Systems,
- Database-System Applications,
- View of Data,
- Database Languages,
- Database System Structure,
- Data Models.
- Database Design and ER Model:
- Entity, Attributes, Relationships, Constraints, Keys, Design Process, Entity-Relationship Model, ER Diagram,
- Design Issues,
- Extended E-R Features,
- converting ER and EER diagram into tables.

Syllabus

Introduction,

Database Users and Administrators,

Need of Database Management Systems,

Characteristics of the Database,

DBMS Architecture,

Data Independence,

View of Data,

Database System Applications,

Data Models:

Relational Model, Hierarchical Model, Network Model, Object-based Model.

- **Unit 2: (ER Modelling and Relational Model) (04 Hours)**
- Entity-Relationship Model: Entities, Attributes, and Relationships; Cardinality of Relationships; Strong and Weak Entity Sets; Generalization, Specialization, and Aggregation; Translating your ER model into a relational model with integrity constraints over relations. Relational Model: Concept of relations, referential integrity, and foreign keys,

- Relational algebra operators: selection, projection, cross product, various types of joins, division, example queries, tuple relational calculus, domain relational calculus, and converting the database specification in E/R notation to the relational schema.

- **Unit 3(SQL and PLSQL) (05 Hours)**
- SQL Basic, DDL, DML, DCL, DTL, Joins and Set Operations, SQL Constraints: Primary Key, Foreign Key, Unique, Not Null, Check, Default, Group by, Order by, Having Clauses, Subqueries, View, and Index. PL/SQL: Concept of Stored Procedures and Functions, Cursors, Triggers, Assertions, Roles, and Privileges.

- Draw architecture of DBMS system and explain function of following
- components: **[9]**
- i) Storage manager
- ii) Query Processor

- Explain the concept of candidate key and primary key, foreign key. Identify above listed key for the following schema: **[6]**
  - Person (driver_id, name, address, contactno)
  - Car (licence, model, year)
  - Owns (driver_id, licence)

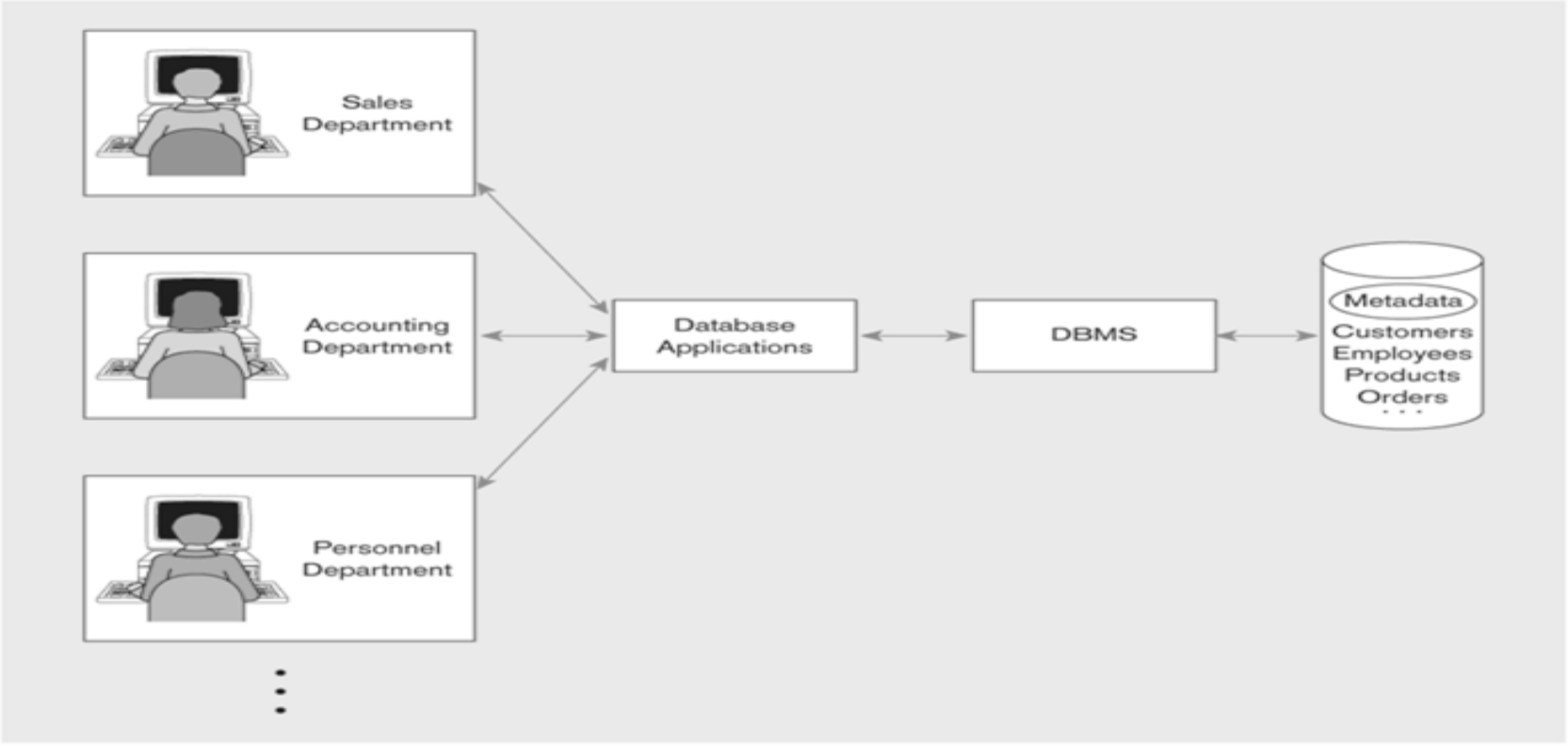
Introduction

❑ *Database Management System:*

- A database management system (DBMS) is *system software for creating and managing databases or data.*
- A DBMS makes it possible for end users to *create, protect, read, update, retrieval, insert and delete* data in a database.
- The DBMS provides a *centralized view of data* that can be *accessed by multiple users* from *multiple locations* in a controlled manner.
- Database systems are *designed to manage large amount of data* by providing ***security*** from *accidental crash of system* and *unauthorized access*.

Database Management System

Figure 1-3 Database approach at Pine Valley Furniture Company



DBMS manages data resources like an operating system manages hardware resources

Introduction Database Management System (DBMS)

- ***DBMS contains information about a particular enterprise:***
  - Collection of interrelated data.
  - Set of programs to access the data.
  - An environment that is both *convenient* and *efficient* to use.

- ***Database Applications:***
  - Banking: transactions
  - Airlines: reservations, schedules
  - Railway reservation Systems
  - Universities/Education Systems: registration, grades
  - Sales: customers, products, purchases
  - Online retailers: order tracking, customized recommendations
  - Manufacturing: production, inventory, orders, supply chain
  - Human resources: employee records, salaries, tax deductions
  - Telecom
  - Library Management Systems.
  - Social media sites

Introduction

- **Purpose of Database Systems:**

- A collection of application programs that perform services for the end-users such as production of reports

- Each program defines and manages its own data

Database Users and Administrators

- **Database User** is defined as a person who interacts with data daily for updating, reading, and modifying the given data.

- Database users can ***access and retrieve data from the database*** through DBMS applications and interfaces.

- Database users are categorized based on their interaction with the database.
- **List:**

1. Database Administrator (DBA)
2. Naive / Parametric End Users
3. A System Analyst
4. Sophisticated Users
5. Database Designers
6. Application Programmers
7. Specialized users

Database Users and Administrators

1. **Database Administrator (DBA):**

- DBA defines the schema and manages all three levels of the database architecture.
- They create user **accounts**, **control access** and **ensure data security.**

- DBAs are also responsible for **handling security breaches** and optimizing system performance.

DBA :

- also monitors the recovery and backup and provides technical support.

- has a DBA account in the DBMS which is called a system or superuser account.

- repairs damage caused due to hardware and/ software failures.

- is the one having privileges to perform DCL (Data Control Language) operations.

Database Users and Administrators

- **Naive / Parametric End Users:**

- Parametric End Users are the **unsophisticated** who lacks DBMS knowledge but they frequently use the database applications in their daily life to get the desired results.

- **For example:**
- 1. **Railway's ticket booking users,**

- 2. **Clerks** in any bank are naive user because they lacks DBMS knowledge but they still use the database.

- **A System Analyst:**

- A system Analyst is a **user who analyzes** the requirements of Naïve users.

- They check whether all the requirements of end users are satisfied.

Database Users and Administrators

- **Sophisticated Users:**

- Sophisticated users like ***engineers*** or analysts ***interact directly with the database using SQL queries.***

- They d**on't write full application code** but use the query processor to access and manipulate data.

- These users **develop custom database applications** based on their specific needs.

Database Users and Administrators

- **Database Designers:**

- Database Designers ***create the structure of a database***, including tables, views, indexes, triggers, and constraints.

- They decide ***what data to store and how it should be related*** based on user requirements.

- Their design is finalized before the database is built or populated with data.

Database Users and Administrators

- **Application Programmers:**

- also known as **System Analysts or Software Engineers**, **write backend code** for application programs.

- They use languages like C, Visual Basic, COBOL,php etc., to design, test, and maintain programs.

- These programs, called "canned transactions," help naive users interact with the database efficiently.

Database Users and Administrators

- **Specialized users:**

- Specialized users are sophisticated users who **write specialized database application** that does not fit into the traditional data-processing framework.

- Among these applications are computer aided-design systems, knowledge-base and expert systems etc.

Traditional approach: File Processing Systems

Traditionally, data was stored and retrieved using files in a typical file system.

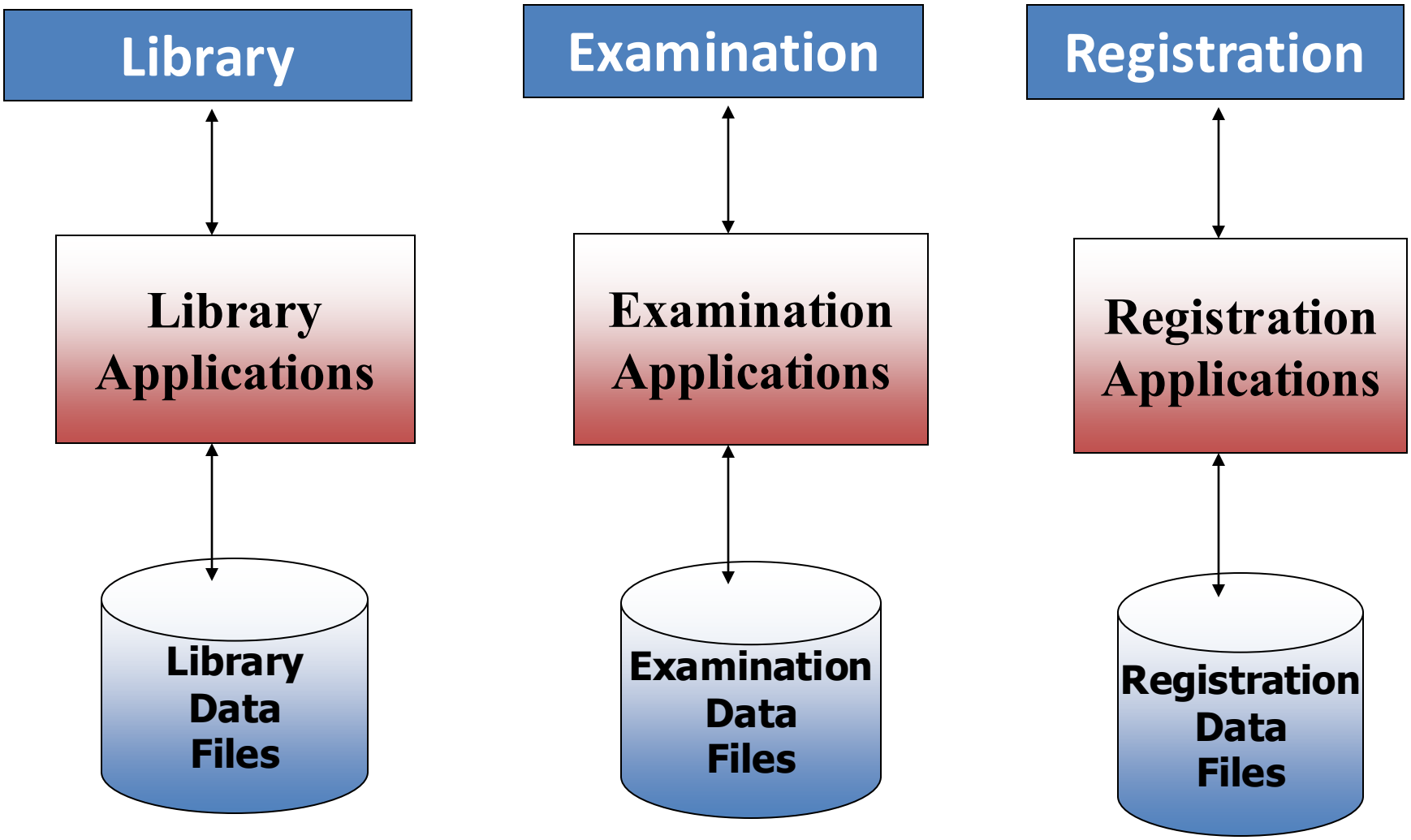
A company might keep separate files for college details, student information and Dept info.

These files could be stored as text documents, spreadsheets or printed records .

This approach ***worked fine for small amounts of data*** but became ***challenging as the volume of data increased.***

Need of Database Management Systems

File Processing Systems



Program and Data Interdependence

File Processing Systems

Library
Reg_Number
Name
Father Name
Books Issued
Fine

Examination
Reg_Number
Name
Address
Class
Semester
Grade

Registration
Reg_Number
Name
Father Name
Phone
Address
Class

Need of Database Management Systems

Traditional approach: File Processing Systems

➢ ***Limitations :***

✓Data Redundancy
✓ Data Inconsistency
✓ Difficulty in Data Retrieval
✓Limited Security
✓No Support for Relationships Between Data
✓Concurrency Issues

To overcome above problems database came into market.

➢ **Need or Advantages of DBMS:**

✓Reduced Data Redundancy

✓Improved Data Integrity and Consistency

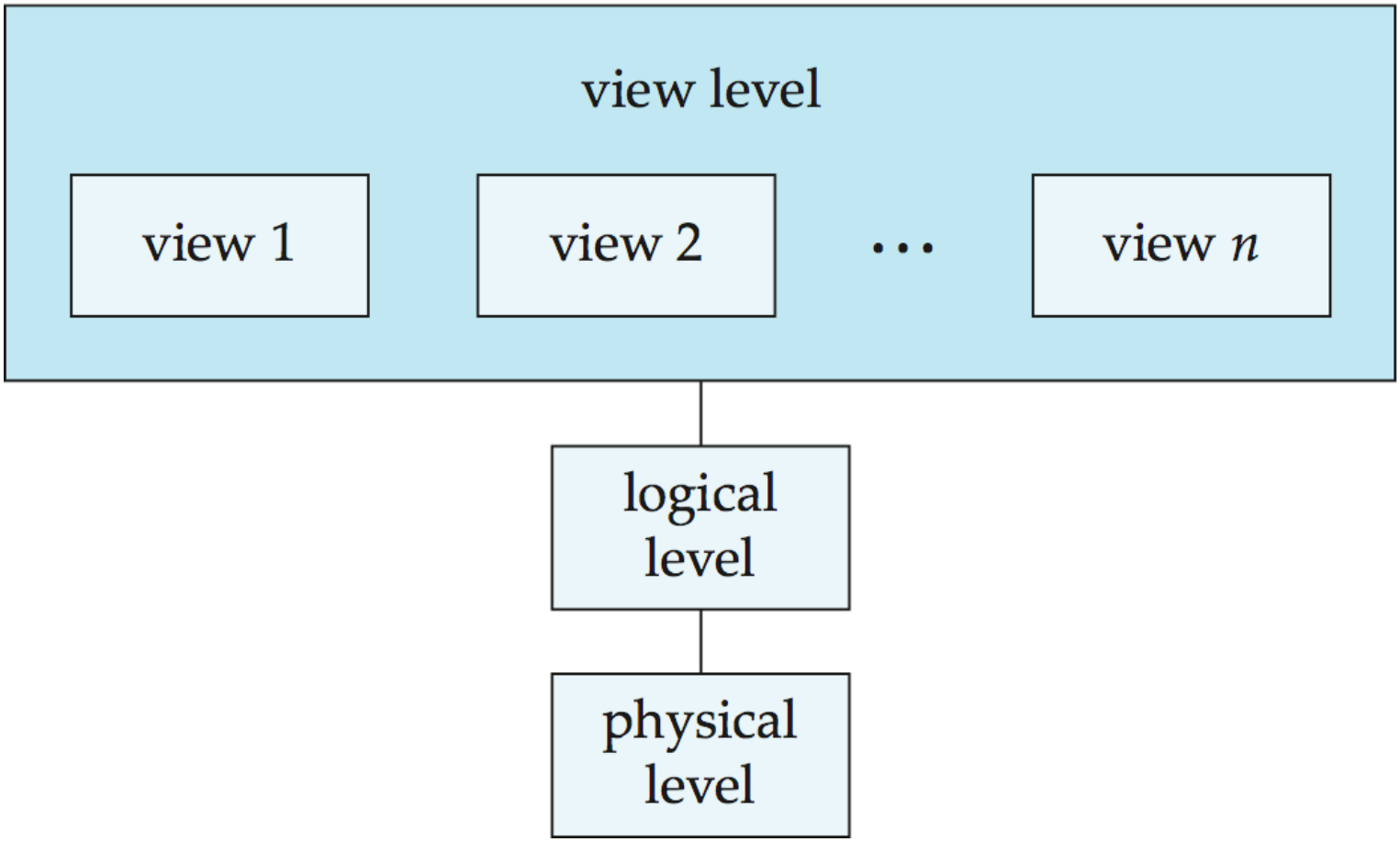
✓Enhanced Security

✓ Simplified Data Access

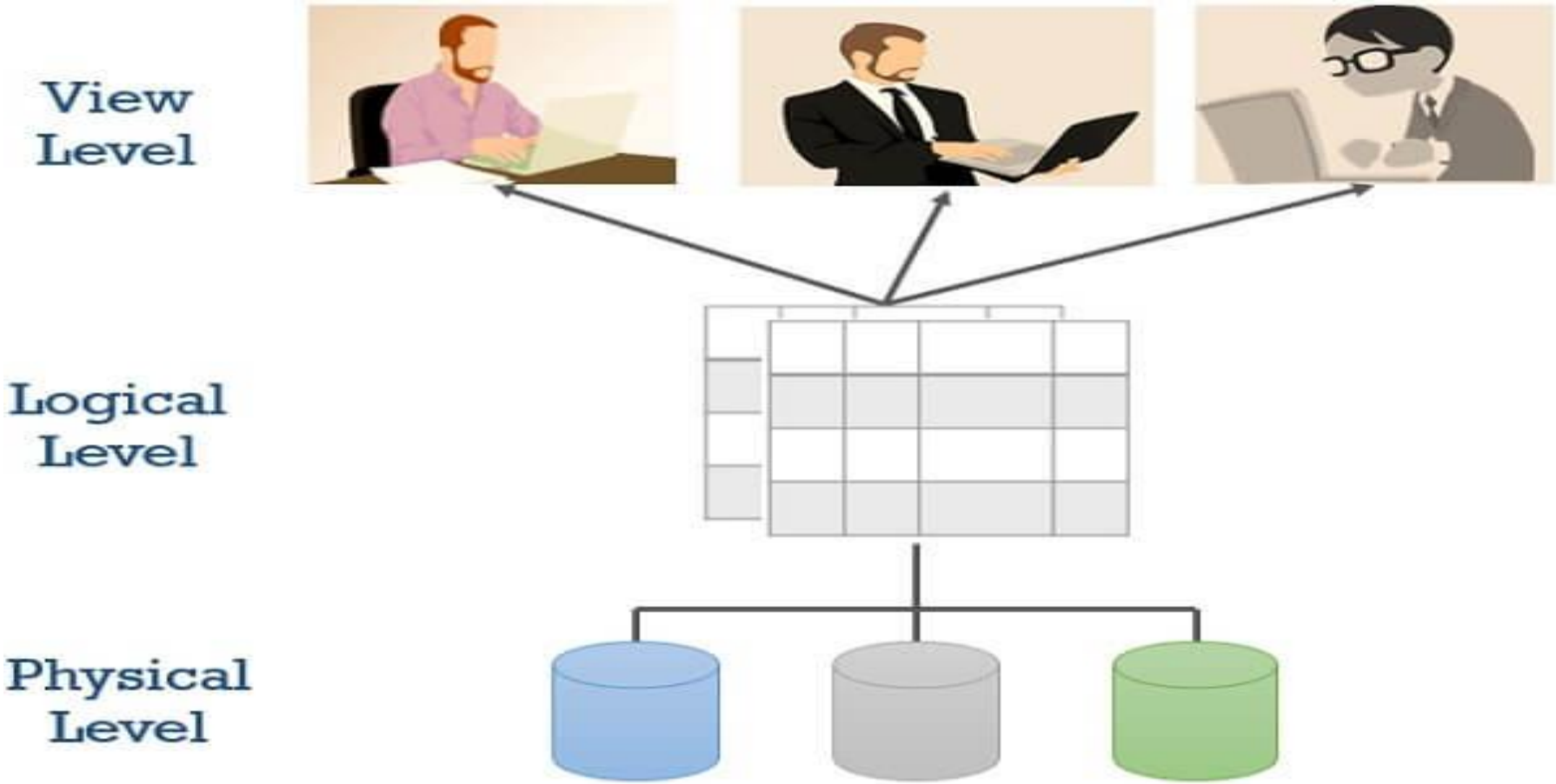
✓Support for Data Relationships

✓Concurrency Control

DBMS Three-Level Architecture



View of Data



Three-Schema Architecture

View of Data

- **Physical level:**

- the internal level schema describes **how the data/record is stored in the hardware**.

- It also describes ***how the data/record can be accessed*****.**

- The physical level shows the ***data abstraction*** at the *lowest level* and it has **complex data structures**.

- Only the ***database administrator*** operates at this level.

View of Data

- **Data abstraction** allow developers to *keep complex data structures **away from the users***.

- The developers achieve this by hiding the complex data structures through **levels of abstraction**.

- A database schema is **the logical representation of a database**, which shows how the data is stored logically in the entire database.

View of Data

- **Logical level/ Conceptual Level :** describes *data stored in database*, and the *relationships* among the data.

- It is a level above the *physical level*.

- the *data is stored* in the form of **entities, entity set**, , their **data types**, the **relationship** among the entity sets.

- **user operations** performed to retrieve or modify the data and certain **constraints on the data**.

- adding *constraints* to the view of data adds the *security*.

- As *users are restricted to access* some particular parts of the database.

View of Data

- **Example:**

type *instructor* = **record**

ID : string;

name : string;

dept_name : string;

salary : integer;

end;

View of Data

- **View level: User level/ External level :**
- application programs *hide details of data types*.
- Views can also *hide information* (such as an employee's salary) for security purposes.
- It display the *data in which the user is interested.*
- The view level can describe *many views of the same data.*
- the *user retrieves* the information using *different application* from the *database*.

Database System Structure

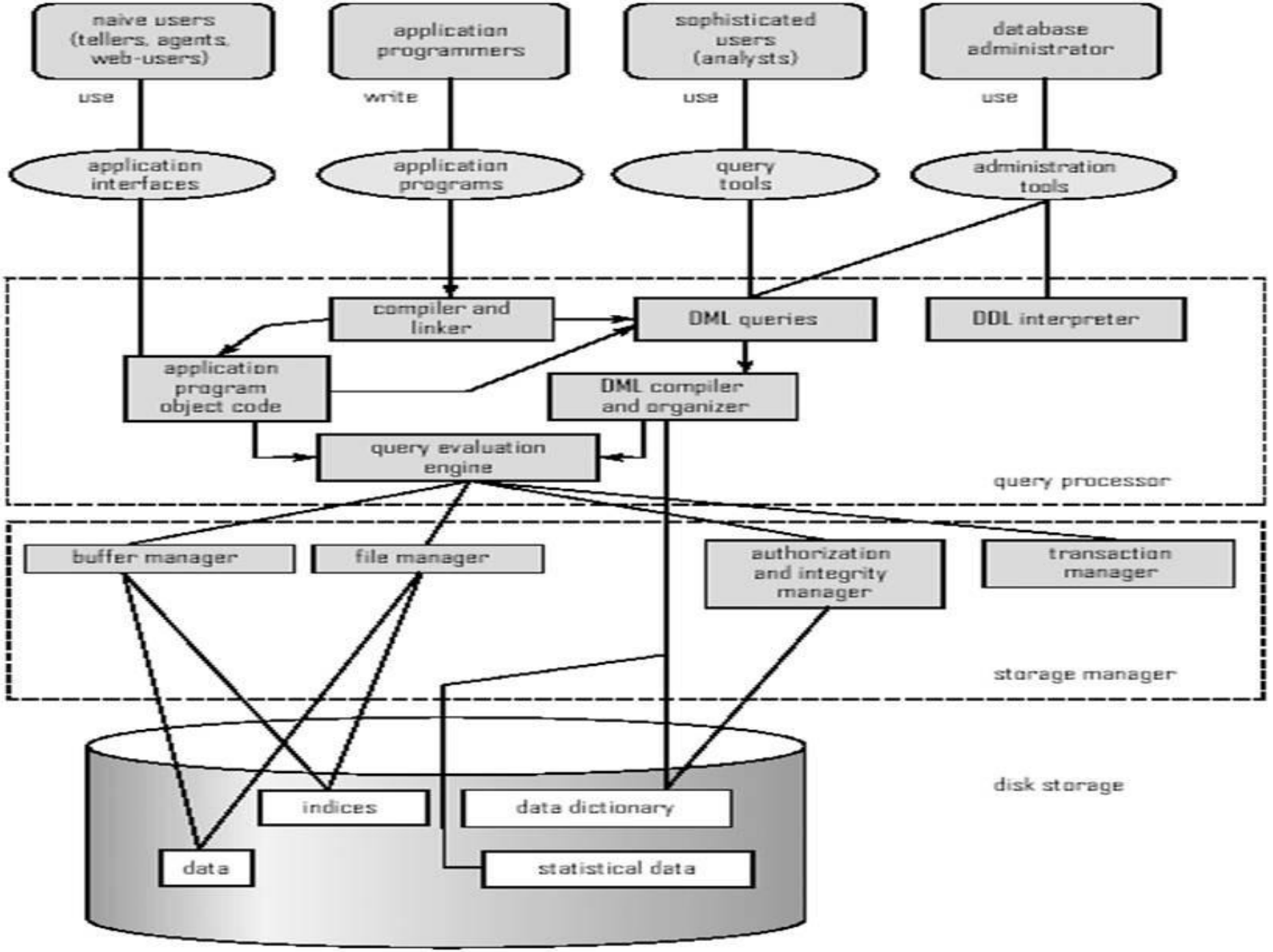
- ***Users of DBMS include:***
  - Application programmers,
  - Database Administrators (DBAs), and
  - end users.

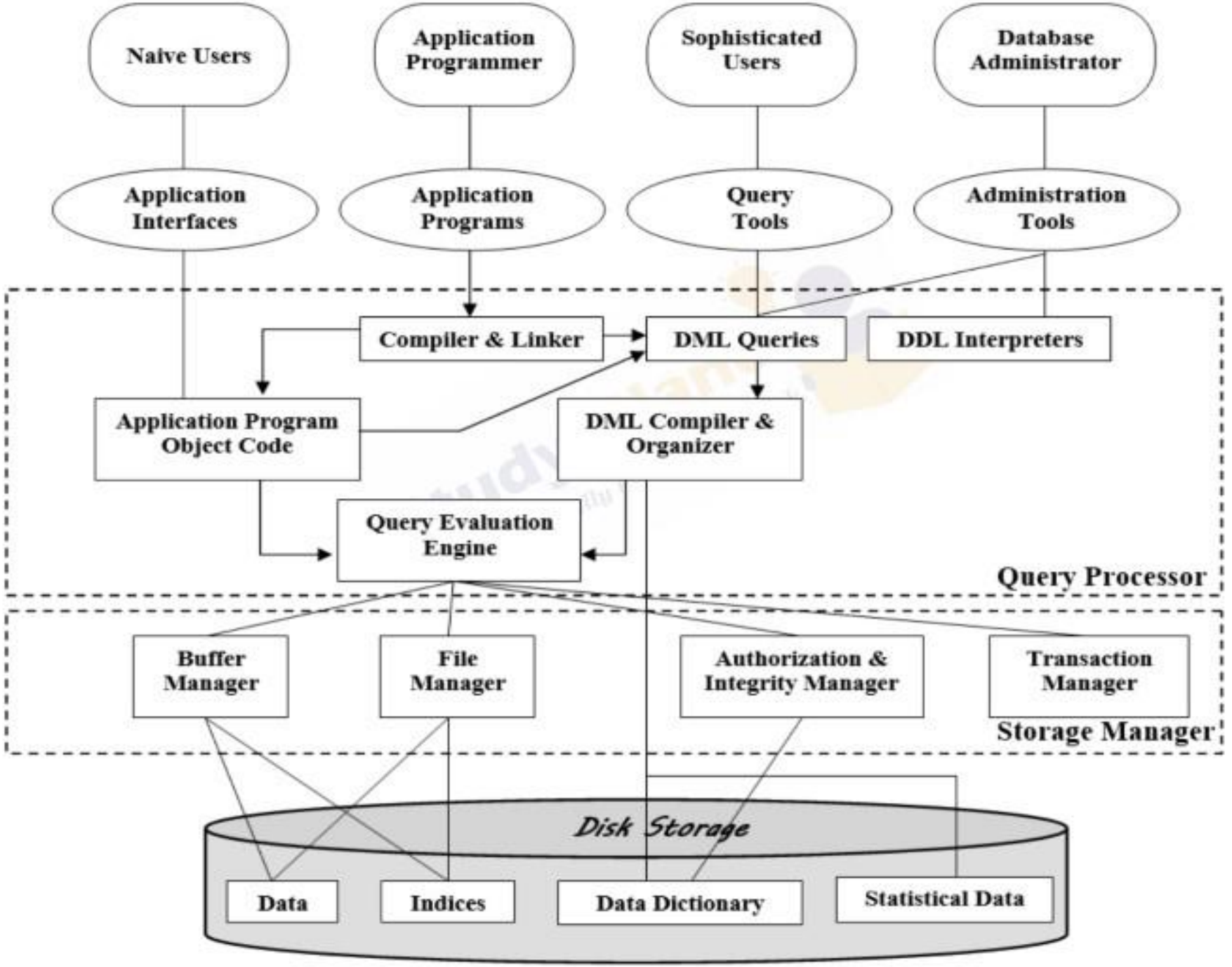
- ***Database Administrators*** are typically the only people *who work directly with a DBMS.*

- *end users* ***read*** and ***write*** to *databases* using ***front-end interfaces*** made by programmers, while *programmers* use cloud APIs to connect with DBMS.

- **Three Parts/components that make up the Database System are:**

  - Query Processor
  - Storage Manager
  - Disk Storage


naive users (tellers, agents, web-users)
application programmers
sophisticated users (analysts)
database administrator
use
write
use
use
application interfaces
application programs
query tools
administration tools
compiler and linker
DML queries
DDL interpreter
application program object code
DML compiler and organizer
query evaluation engine
query processor
buffer manager
file manager
authorization and integrity manager
transaction manager
storage manager
indices
data dictionary
data
statistical data
disk storage


Naive Users
Application Programmer
Sophisticated Users
Database Administrator
Application Interfaces
Application Programs
Query Tools
Administration Tools
Compiler & Linker
DML Queries
DDL Interpreters
Application Program Object Code
DML Compiler & Organizer
Query Evaluation Engine
Query Processor
Buffer Manager
File Manager
Authorization & Integrity Manager
Transaction Manager
Storage Manager
Disk Storage
Data
Indices
Data Dictionary
Statistical Data

Database System Structure

- **Three Parts/Components that make up the Database System are:**

1. Query Processor
2. Storage Manager
3. Disk Storage

1. Query Processor:

Components:

- ✓ DDL Interpreter
- ✓ DML Compiler
- ✓ Embedded DML Pre-compiler
- ✓ Query Optimizer

Database System Structure

- **Three Parts that make up the Database System are:**

2. Storage Manager:

- ✓ Integrity Manager
- ✓ Authorization Manager
- ✓ File Manager
- ✓ Transaction Manager
- ✓ Buffer Manager

3. Disk Storage

- ✓ Data Dictionary
- ✓ Data Files
- ✓ Indices

Database System Structure

1. Query Processor:

- Its primary duty is to ***successfully execute the query.***
- It transforms (or interprets) the *user's application program-provided requests* into *instructions* that a computer can understand.
- It executes the ***user's query*.**
- the query processor helps the database system in making *data access simple and easy.*
- It receives ***input query** in the form of SQL text* and ***optimizes them*** and *completes their execution* by employing specific ***data access methods*** and ***database operator*** implementation .

Database System Structure

Components:

✓ **DDL Interpreter:**

- DDL Interpreter ***interprets DDL statements*** like those used ***in schema definitions*** (such as create, drop, etc.).

- This interpretation produce a ***set of tables*** that include ***the meta-data*** that is kept in the ***data dictionary***.

- Metadata may be stored in a ***data dictionary***.

✓ **DML compiler:**

- the DML Compiler converts ***DML statements*** like select, update, and delete into ***low-level instructions*** or simply ***machine-readable object code***, to enable execution.

Database System Structure

- The ***optimization of queries*** is another function of the DML compiler.

- Since a ***single question*** can typically be ***translated*** into a ***number of evaluation plans.***

- As a ***result***, some optimization is needed to ***select the evaluation plan*** with the ***lowest cost*** out of all the options.

- This process, known as ***query optimization***.

✓ **Embedded DML Pre-compiler:**

- Before the query evaluation, the ***embedded DML commands*** in the application program (such as SELECT, FROM, etc., in SQL) must be ***pre-compiled*** into standard ***procedural calls.***

Database System Structure

✓ **Query Optimizer:**

- It starts by taking the ***evaluation plan for the question***, runs it, and then returns the result.

- The ***query evaluation engine*** evaluates the ***SQL commands*** used to access the database's contents before returning the result of the query.

Database System Structure

2. Storage Manager:

- An application called Storage Manager acts as a ***mediator*** between ***the queries made*** *and* ***the data kept in the database****.*
- it is also called as ***Database Control System***.
- By applying the ***restrictions*** and running the ***DCL instructions***, *it keeps the consistency and integrity of database.*
- It is ***in charge of*** retrieving, storing, updating, and removing data from the database.

Database System Structure

- **Components of Storage Manager:**

✓ **Integrity Manager:**

- Whenever there is ***any change in the database***, the Integrity manager will *manage the integrity constraints.*

✓ **Authorization Manager:**

- Authorization manager ***verifies*** the ***user*** that he is ***valid*** and ***authenticated*** for the specific query or request.

✓ **File Manager:**

- All the ***files and data structure of the database*** are managed by this component.

Database System Structure

✓ **Transaction Manager:**

- It is responsible for ***making the database consistent*** before and after the transactions.

- ***Concurrent processes*** are generally controlled by this component.

✓ **Buffer Manager:**

- The ***transfer of data*** between ***primary*** and ***main memory*** and ***managing the cache memory*** is done by the buffer manager.

Database System Structure

- **3. Disk Storage**

- A DBMS can use various kinds of ***Data Structures*** ***as a part of physical system implementation*** in the form of disk storage.

✓ **Data Dictionary:**

- It contains the ***metadata*** (data of data), which means *each object of the database* has *some information about its structure*.

- So, it *creates a repository* which contains the details about the structure of the database object.

✓ **Data Files:**

- This component ***stores the data*** in the files.

✓ **Indices:**

- These indices are used to ***access*** *and* ***retrieve*** *the data in a very fast and efficient* way.

Data Independence

➢ Data Independence

- Data independence in a DBMS refers to the ability to modify the database schema at one level of abstraction without affecting the schema at other levels.
- **OR**
- the ability to modify the schema at one level of the database without affecting the schema at the next higher level.

- ***Data independence exists between these levels:***

- **Physical Data Independence:** Change in the internal level without affecting the logical level.

- **Logical Data Independence:** Change in the logical level without affecting the view level.

Data Independence

- **Types of Data Independence:**

- Logical Data Independence
- Physical Data Independence
- **Explanation:**

✓ **Logical Data Independence:**

- Ability to change the logical structure (tables, columns, relationships) without affecting external views or application programs.

- Purpose of this to allow the database structure to evolve without impacting user access or requiring changes in application code.

- **Example:**
  - *Adding a column like class in the student table.*
  - *Creating a new relationship between two tables.*
  - *Merging two tables into a view for simplified access.*

Data Independence

- **Physical Data Independence:**

- The ability to change how data is physically stored without affecting the logical schema or user-facing applications.

- Purpose is to improve performance, storage efficiency, or hardware configurations without changing how the data is structured logically.

- **Example:**
  - *Moving data files from the C: drive to the D: drive.*
  - *Creating an index to speed up queries.*
  - *Compressing data files to save space.*

Data Independence

✓ **Logical Data Independence:**

- **Benefits:**
- Easier maintenance of application code.
- Allows smoother updates in growing systems.
- Helps support new business requirements without rewriting existing queries.

✓ **Physical Data Independence:**

- **Benefits:**
- Backend optimizations without affecting users
- Reduces need for structural changes during storage upgrades
- Improves long-term maintainability

Data Models

- Data Model is the ***modeling of the data description***, ***data semantics*** and ***consistency constraints*** of the data.
- It provides the ***conceptual tools*** for describing the design of a database at each level of data abstraction.
- It is a ***collection of tools*** used for describing data, relationships among them, data semantics and consistency constraints.
- Process of analysis of data object and their relationship to other data objects is know as data model.

Data Models: Types

➢ Types of data models

- **Types of data models** used for understanding the structure of the database:
  - Relational Model
  - Entity relationship model
  - Object oriented model
  - Hierarchical model
  - Network Model
  - Physical Data Model

Data Models

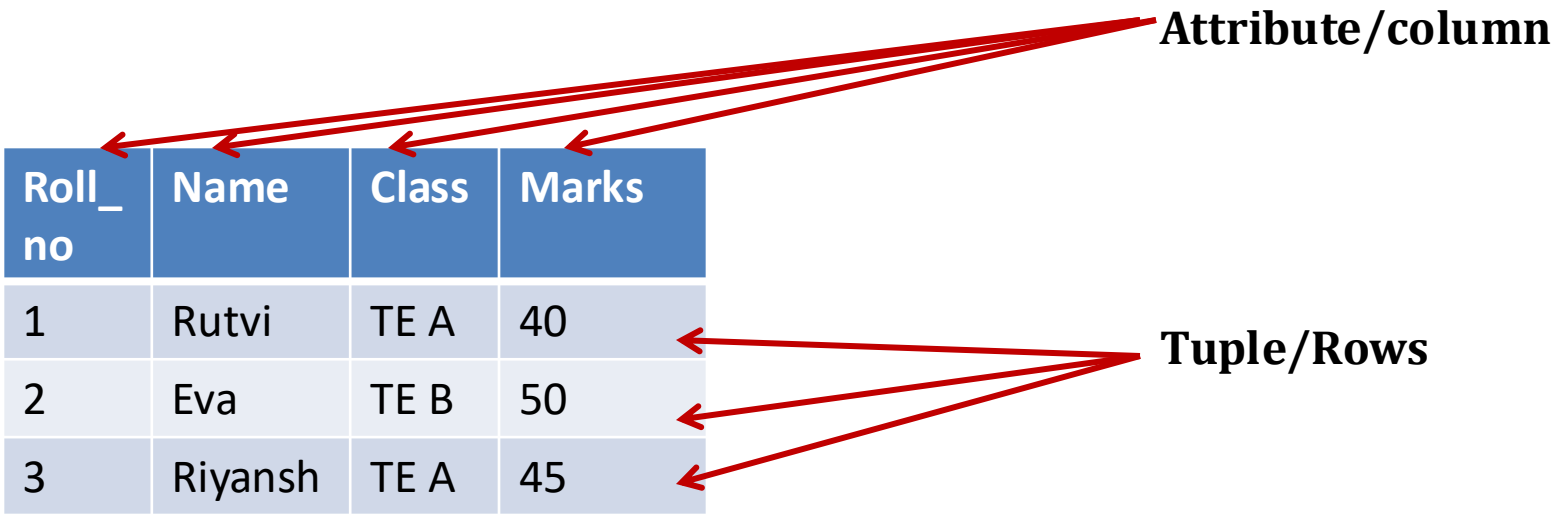
1. **Relational Data Model:** (Note: can refer the unit III ppt first point)

- E.F. codd proposed the ***relational model*** to ***model data*** in the form of relations or tables.
- It represents ***how data is stored*** in relational databases.
- a relational model uses ***tables*** for representing data and in-between relationships.
- This type of model designs the data ***in the form of rows and columns*** within a table.
- ***Tables*** are also called ***relations***. **Columns** are called **Attribute** , Rows or records are called as Tuple
- The relational data model is the widely used model which is ***primarily used by commercial data processing applications*****.**

Data Models

1. Relational Data Model:

- This model is called as record based model because the database is structured in fixed format.
- Every record must have uniquely identified by the key.
- **Example**: Consider Table **Student**



Roll_no	Name	Class	Marks
1	Rutvi	TE A	40
2	Eva	TE B	50
3	Riyansh	TE A	45

Data Models

2) Entity-Relationship Data Model: (Note*: Database Design and ER Model*)

- An ER model is the *logical representation of data* as objects and *relationships* among them.
- These objects are known as entities, and relationship is an association among these entities.
- This model was designed by Peter Chen and published in 1976 papers.
- It was widely used in *database designing*.
- A set of attributes describe the entities.
- **Example:** student_name, student_id describes the 'student' entity.
- A set of the same type of entities is known as an ***'Entity set***', and the set of the same type of relationships is known as ***'relationship set***'.

Data Models

2) Entity-Relationship Data Model: (Note*: Database Design and ER Model*)

- An Entity Relationship Diagram (ER Diagram) pictorially explains the relationship between entities to be stored in a database.

- **Advantages:**

✓ Simple design

✓ Effective representation

✓ Connected with relational model

Data Models

3) Object-Oriented Data Model:

- It is ***extension to the ER model*** with integration of concepts like object, method & encapsulation,
- data and their relationships are contained in a single structure which is referred to as an object in this data model.
- real-world problems are represented as ***objects*** with different attributes.
- All objects have multiple relationships between them.
- Basically, it is a ***combination*** of ***Object Oriented programming*** and a **Relational Database Model**.
- It is the collection of ***objects*** or ***elements*** with its methods.
- It supports inheritance, encapsulation.

Data Models

4) Hierarchical model:

- Data are organized in ***tree like structure*** with one root.
- Data is get arrnged in parents-child relationship structure.
- ***Highest root node*** will parent & other nodes will be children.
- It supports ***one to many relationship*** i. e. only one parent with many Childs.
- Main ***application*** : mainframe database system.
- It is another type of ***record based model***.
- i.e data is stored in the form of records. These records are connected to one another.
- Records are the collection of fields. Each field contains only one value.

Data Models

4) Hierarchical model:

- **Advantages:**

✓ Easy addition and deletion of node

✓ Data at top is fast to access

✓ Simple to understand

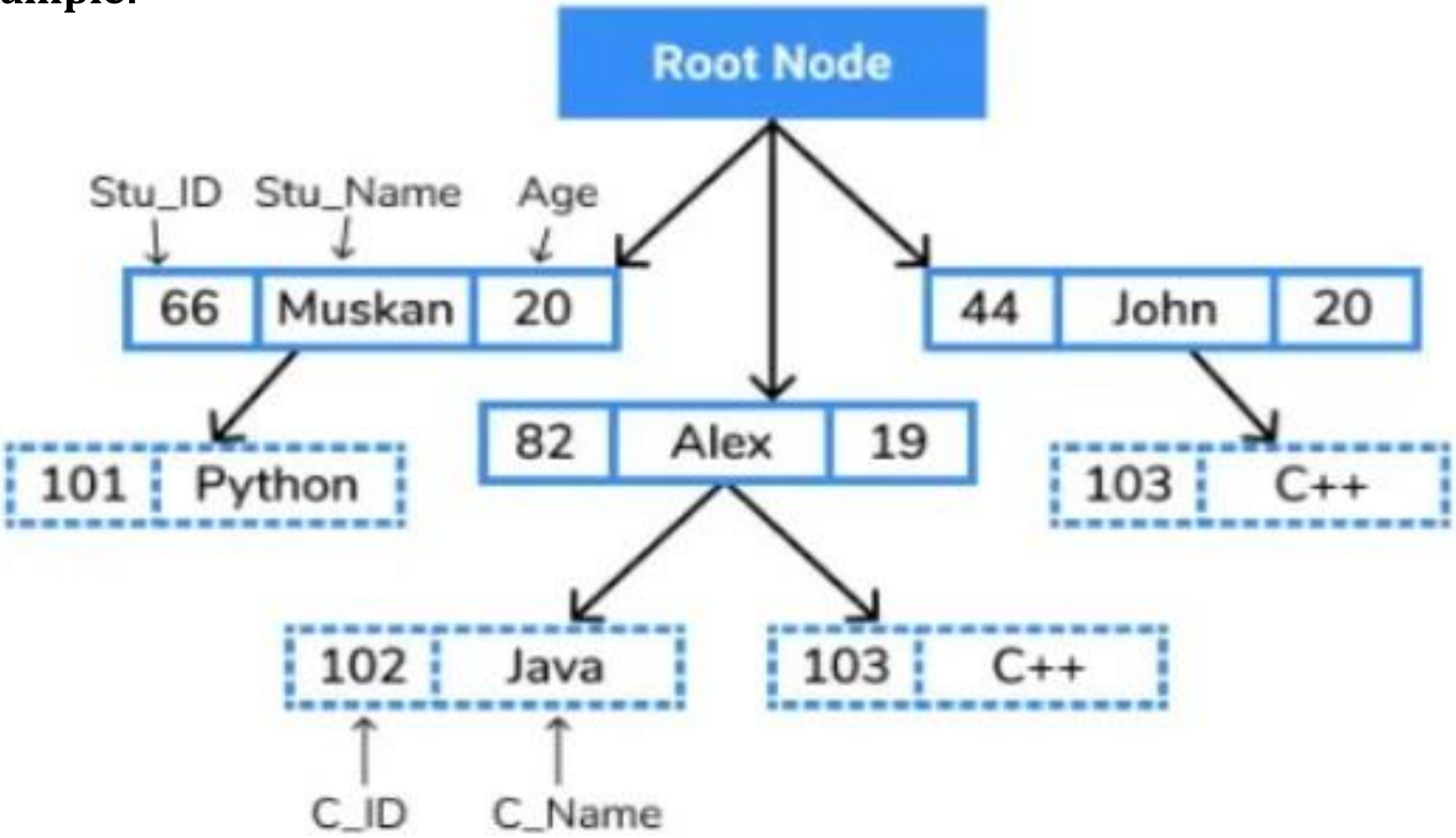
✓ Database Integrity

✓ Efficiency

Data Models

4) Hierarchical model:

Example:



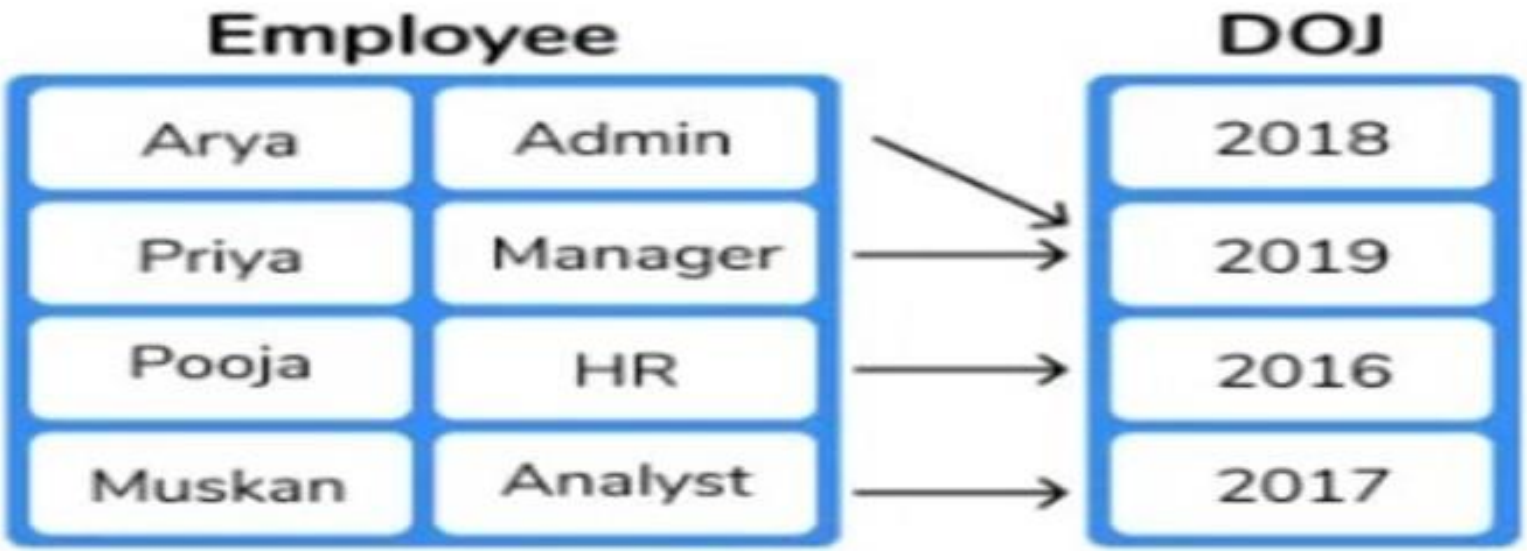
Data Models

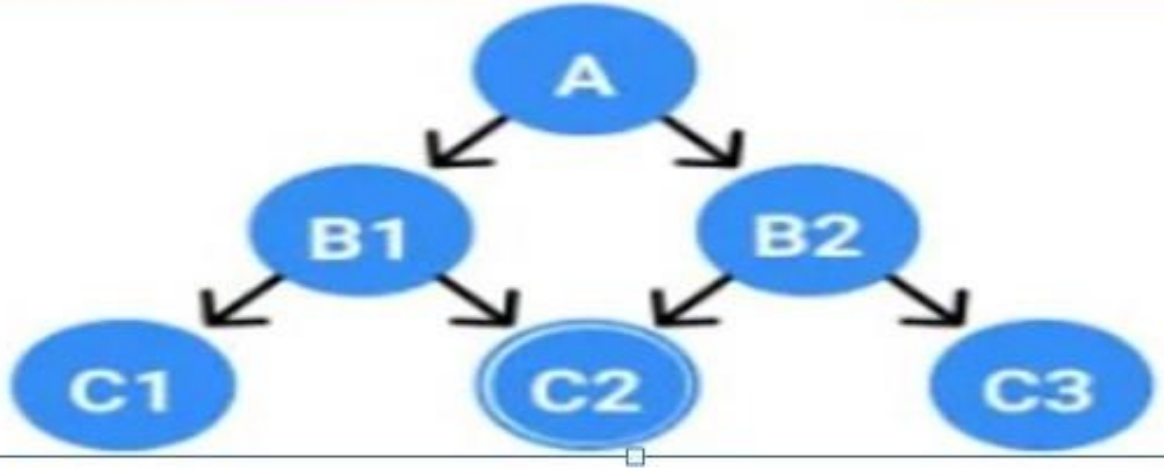
4)Network Model:

- Network model which is an extension to the hierarchical model
- Data is represented *by collection of records* & *relationships* among data are represented by *links*.
- Data Organization in the network model is just like a ***graph*** rather than a tree.
- It allows a record to have *more than one parents of many child*.
- It represents ***one to many*** or **many to many** relationship.
- It is tree like structure with many roots.
- Less popular than hierarchical model.
- It uses ***client server architecture***.
- Introduced by charls Bachman.

Data Models

4)Network Model:





Data Models

4)Network Model:

- **Advantages**:

- **Ability to handle more relationship type:** one to many or many to many
- **Simplicity**: Conceptually simple and easy to design.
- **Data integrity**: members can not exist without owner record.

  User must firstly define the owner record & then the member record. This ensures data integrity.

- **Disadvantages:**

- system complexity
- Lack of structural independence